Synchronization

Synchronization in POSIX

Stefano Quer
Dipartimento di Automatica e Informatica
Politecnico di Torino

License Information

This work is licensed under the license

CC BY-NC-ND 4.0

Attribution-NonCommercial-NoDerivatives 4.0 International

This license requires that reusers give credit to the creator. It allows reusers to copy and distribute the material in any medium or format in unadapted form and for noncommercial purposes only.

BY: Credit must be given to you, the creator.

NC: Only noncommercial use of your work is permitted.
Noncommercial means not primarily intended for or directed towards commercial advantage or monetary compensation.

ND: No derivatives or adaptations of your work are permitted.

To view a copy of the license, visit:
https://creativecommons.org/licenses/by-nc-nd/4.0/?ref=chooser-v1

Introduction

- Semaphores are a very specific form of Inter-Process Communication
  - They correspond to a counter protected by a lock (i.e., a binary semaphore or lock) with a waiting queue
  - To implement a semaphore, manipulation functions must be atomic operations
    - For this reason, semaphores are normally implemented inside the kernel
  - It is possible to use synchronization strategies faster and less expensive than semaphores (e.g., mutexes)

POSIX semaphores

- POSIX system calls are included in the header file
  - semaphore.h
- A semaphore is a type sem_t variable
  - sem_t *sem1, *sem2, ...;
- All semaphore system calls
  - Have name sem_*
  - On error, they return the value -1

System calls:
sem_init
sem_wait
sem_trywait
sem_post
sem_getvalue
sem_destroy

sem_init

```
int sem_init (
   sem_t *sem,
   int pshared,
   unsigned int value
);
```

- Initializes the semaphore counter at value **value**
- The value **pshared** identifies the semaphore type
  - If equal to `0`, the semaphore is local to the **threads of the current process**
  - Otherwise, the semaphore can be **shared between different processes** (parent that initializes the semaphore and its children)

Linux does not currently support shared semaphores

sem_wait

```
int sem_wait (
   sem_t *sem
);
```

- Standard wait
  - If the semaphore is equal to 0, it blocks the caller until it can decrease the value of the semaphore

sem_trywait

```
int sem_trywait (
   sem_t *sem
);
```

- Non-blocking wait
  - If the semaphore counter has a value greater than 0, perform the decrement, and returns 0
  - If the semaphore is equal to 0, returns -1 (instead of blocking the caller as **sem_wait** does)

sem_post

```
int sem_post (
   sem_t *sem
);
```

- Standard signal
  - Increments the semaphore counter, or wakes up a blocked thread if present

sem_getvalue

```
int sem_getvalue (
   sem_t *sem,
   int *valP
);
```

Better not use this function. From Linux manual: "The value of the semaphore may already have changed by the time sem_getvalue() returns"

- Allows obtaining the value of the semaphore counter
  - The value is assigned to *valP
  - If there are waiting threads
    - 0 is assigned to *valP (Linux)
    - or a negative number whose absolute value is equal to the number of processes waiting (POSIX)

sem_destroy

```
int sem_destroy (
   sem_t *sem
);
```

- Destroys the semaphore at the address pointed by sem
  - Destroying a semaphore that other threads are currently blocked on produces undefined behavior (on error, -1 is returned)
  - Using a semaphore that has been destroyed produces undefined results, until the semaphore has been reinitialized

Esempio

Use of sem_* primitives to synchronize threads

```
#include "semaphore.h"

sem_t sem;
sem_init (&sem, 0, 0);

... create threads ...

sem_destroy (&sem);
```

Static semaphore

```
sem_wait (&sem);
... SC ...
sem_post (&sem);
```

Dynamic semaphore

```
#include "semaphore.h"

sem_t *sem;
sem = (sem_t *)
      malloc(sizeof(sem_t));
sem_init (sem, 0, 0);

... create threads ...

sem_destroy (sem);
```

```
sem_wait (sem);
... SC ...
sem_post (sem);
```

Pthread mutex

- A **mutex** is basically a lock that we
  - Set (lock) before accessing a shared resource
    - While it is set, any other thread that tries to set it will block until we release it
  - Release (unlock) when we are done
    - If more than one thread is blocked when we unlock the mutex, then all threads blocked on the lock will be made runnable, and the first one to run will be able to set the lock

```
while (TRUE) {
   ...
   reservation code
   Critical Section
   release code
   ...
   non critical section
}
```

Classical Critical Section protocol

Pthread mutex

- Mutexes are essentially
  - **Binary**, **local** and **fast** semaphores
- A mutex
  - Is a variable represented by the **pthread_mutex_t** data type
  - It can be managed using the following system calls
    - pthread_mutex_init
    - pthread_mutex_lock
    - pthread_mutex_trylock
    - pthread_mutex_unlock
    - pthread_mutex_destroy

A mutex is less general than semaphores (i.e., it can assume only the two values 0 or 1)

pthread_mutex_init

- Before we can use a mutex variable, we must first initialize it by
  - Either setting it to the constant PTHREAD_MUTEX_INITIALIZER, for statically allocated mutexes only and default attributes
  - Calling **pthread_mutex_init**, if we allocate the mutex dynamically (e.g., by calling **malloc**) or we want to set specific attributes
    - If we dinamically allocate it, then we need to call **pthread_mutex_destroy** before freeing the memory (i.e., by calling **free**)

pthread_mutex_init

```
int pthread_mutex_init (
  pthread_mutex_t *mutex,
  const pthread_mutexattr_t *attr
);
```

Always include pthread.h

- Initializes the mutex referenced by **mutex** with attributes specified by **attr**
  - To initialize a mutex with the default attributes, we set **attr** to NULL
- Return value
  - The value, 0 on success
  - An error code, otherwise

pthread_mutex_lock

```
int pthread_mutex_lock (
  pthread_mutex_t *mutex
);
```

- Lock a mutex, i.e., control its value and
  - Blocks the caller if the mutex is locked
  - Acquire the mutex lock if the mutex is unlocked
- Return value
  - The value 0, on success
  - An error code, otherwise

pthread_mutex_trylock

- If a thread can't afford to block, it can use pthread_mutex_trylock to lock the mutex conditionally
  - If the mutex is unlocked at the time **pthread_mutex_trylock** is called, then pthread_mutex_trylock will lock the mutex without blocking and return 0
  - Otherwise, **pthread_mutex_trylock** will fail, returning EBUSY without locking the mutex

pthread_mutex_trylock

```
int pthread_mutex_trylock (
  pthread_mutex_t *mutex
);
```

- Similar to pthread_mutex_lock, but returns without blocking the caller if the mutex is locked
- Return value
  - The value 0, if the lock has been successfully acquired
  - **EBUSY** error if the mutex was already locked by another thread

pthread_mutex_unlock

```
int pthread_mutex_unlock (
  pthread_mutex_t *mutex
);
```

- Release (unlock) the mutex lock (typically at the end of a sritical section)
- Return value
  - The value 0, on success
  - An error code, otherwise

pthread_mutex_destroy

```
int pthread_mutex_destroy (
  pthread_mutex_t *mutex
);
```

- Free **mutex** memory
  - Used for dynamically allocated mutexes
  - The mutex cannot be used anymore
- Return value
  - The value 0, on success
  - An error code, otherwise

Example

Use a dynamically allocated mutex to protect a CS

```
pthread_mutex_t *lock;
...
lock = malloc (1 * sizeof (pthread_mutex_t));
if (lock == NULL) ... error ...
if (pthread_mutex_init (lock, NULL) != 0) ... error ..

...
pthread_mutex_lock (lock);
CS
pthread_mutex_unlock(lock);
...

pthread_mutex_destroy (lock)
free (lock);
```